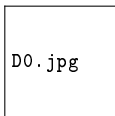


Lecture 32: Advanced SQL

CTEs and Recursive Queries

Z2004: Database Management Systems



IIT Madras Zanzibar

Friday, 5 June 2026

Lecture Objectives

- ▶ Write and read queries using **CTEs** with **WITH**.
- ▶ Explain why CTEs improve **readability** and enable **step-by-step** SQL.
- ▶ Write a **recursive CTE** with an anchor member and a recursive member.
- ▶ Use recursive CTEs to query a **hierarchy** (org-chart) with levels.

The Subquery Mess

Why standard SQL reads inside-out

What if we want to find students whose GPA is higher than the average GPA of the 'CSE' department?
Traditionally, we embed a subquery right into our outer parameters.

```
SELECT Name, GPA
FROM Student
WHERE GPA > (
    SELECT AVG(GPA)
    FROM Student
    WHERE Dept = 'CSE'
);
```

- ▶ **The Human Problem:** As you string together multiple metrics, your code becomes nested and unreadable.
- ▶ **The Architecture Solution:** Instead of writing logical expressions from the inside out, can we establish clean, isolated query blocks from top-to-bottom?

Enter the CTE (Common Table Expression)

The WITH Clause

A **CTE** acts like a temporary, named result block that exists exclusively for the execution scope of that single query. Think of it as a localized variable for a single calculation.

```
-- 1. Initialize the statement using WITH
WITH CseAverage AS (
    SELECT AVG(GPA) AS AvgGPA
    FROM Student
    WHERE Dept = 'CSE'
)
-- 2. Reference the block instantly below
SELECT s.Name, s.GPA
FROM Student s
CROSS JOIN CseAverage c
WHERE s.GPA > c.AvgGPA;
```

CTE vs Permanent View: Views are cataloged directly into database metadata permanently. A CTE evaporates from volatile memory the exact millisecond your server finishes printing out your data!

Example: CTE to find above-average students (by dept)

```
WITH DeptAvg AS (  
    SELECT Dept, AVG(GPA) AS AvgGPA  
    FROM Student  
    GROUP BY Dept  
)  
SELECT s.ID, s.Name, s.Dept, s.GPA  
FROM Student s  
JOIN DeptAvg d ON d.Dept = s.Dept  
WHERE s.GPA > d.AvgGPA;
```

- ▶ **Step 1 (CTE):** DeptAvg computes (Dept, AvgGPA) for each department.
- ▶ **Step 2 (join):** each student *s* is matched to their dept average *d* using *d.Dept = s.Dept*.
- ▶ **Step 3 (filter):** keep only students with *s.GPA > d.AvgGPA* (above their own dept average).
- ▶ Equivalent to a correlated subquery; often clearer and easier to debug. Optimizer behavior (inline vs materialize) is DB-dependent.

Multiple CTEs (building a pipeline)

```
WITH
CSE AS (
    SELECT ID, Name, GPA
    FROM Student
    WHERE Dept = 'CSE'
),
Honor AS (
    SELECT *
    FROM CSE
    WHERE GPA >= 3.5
)
SELECT *
FROM Honor
ORDER BY GPA DESC;
```

- ▶ **Why multiple CTEs?** Break one complex query into named steps.
- ▶ **CTE 1:** CSE creates a temporary table of only CSE students with columns (ID, Name, GPA).
- ▶ **CTE 2:** Honor filters CSE again to keep only rows with GPA ≥ 3.5 .
- ▶ **Final query** selects from Honor exactly like selecting from a real table.
- ▶ **ORDER BY** sorts the remaining students by GPA (highest first).
- ▶ **Key idea:** each CTE exists only for this one statement (not saved like a view).

2. Recursive CTEs (hierarchies / trees)

Structure:

```
WITH RECURSIVE cte AS (  
    -- Anchor member (start rows)  
    SELECT ...  
  
    UNION ALL  
  
    -- Recursive member (adds next "layer")  
    SELECT ...  
    FROM cte  
    WHERE ...  
)  
SELECT *  
FROM cte;
```

- ▶ **Use when:** rows reference other rows (manager, parent, prerequisite, etc.).
- ▶ **Anchor:** produces the starting set (roots / level 0).
- ▶ **Recursive step:** reads the previous results (cte) and produces the next level.
- ▶ **UNION ALL:** appends new rows each iteration (don't use UNION unless you need de-duplication).
- ▶ **Stops when:** the recursive step returns 0 new rows (or a DB recursion limit is hit).

How the recursion actually runs

- 1 Run the **anchor query** once → put rows into a working table.
- 2 Repeat:
 - feed the *new rows from the last round* into the recursive query,
 - append any newly found rows to the result,
 - stop when nothing new is found.
- 3 **Common pitfalls:** cycles in the data (A manages B manages A), missing termination condition, or hitting a recursion-depth limit.

Tip: Always add a `Level` column while learning/debugging, and think: *What is my anchor? What is my next-step join? When does it stop?*

Org-chart / hierarchy example

Table: Employee(EmpID, Name, ManagerID) where ManagerID references EmpID.

```
WITH RECURSIVE Org AS (  
    -- Anchor: start from the CEO (no manager)  
    SELECT EmpID, Name, ManagerID, 0 AS Level  
    FROM Employee  
    WHERE ManagerID IS NULL  
  
    UNION ALL  
  
    -- Recursive: add direct reports  
    SELECT e.EmpID, e.Name, e.ManagerID, o.Level + 1 AS Level  
    FROM Employee e  
    JOIN Org o ON e.ManagerID = o.EmpID  
)  
SELECT *  
FROM Org  
ORDER BY Level, ManagerID, EmpID;
```

- ▶ **Anchor member:** picks the starting node(s) (root of the tree).
- ▶ **Recursive member:** repeatedly expands one level at a time.
- ▶ Most DBs include a recursion limit / safety mechanism.
- ▶ Always think about **cycles** (bad data can cause infinite loops).

CTEs at a glance (why WITH helps)

Technique	What it is	Lifetime / scope	Best for (typical use)
Subquery	Query nested inside another query	Only where it appears	Small, one-off computations; can get hard to read when deeply nested
View	Saved query with a name	Permanent (stored in catalog)	Reuse across many queries; security (hide columns/rows); simplify repeated joins
CTE (WITH)	Named result block inside one statement	Only for that statement	Step-by-step/pipeline queries; readability; breaking complex logic into chunks
Recursive CTE	CTE that refers to itself via UNION ALL	Only for that statement	Hierarchies/trees (org charts, parent-child), graph traversal, generating sequences

Rule of thumb: If you need *re-use across queries*, make a **view**. If you need *clarity inside one query*, use a **CTE**. If the data is a *tree*, use a **recursive CTE**.

In-class Quiz: CTEs (WITH)

Write the exact SQL code (SchoolDB).

Tables: *Student*(SID, Name, Dept, GPA), *Course*(CID, CName, Credits), *Enrolment*(SID, CID, Grade).

- 1 Use a CTE DeptAvg(Dept, AvgGPA) to compute average GPA per department.
- 2 Using that CTE, list SID, Name, Dept, GPA of students whose GPA is above their department average.
- 3 Write the same query again, but with **two CTEs**: first DeptAvg, then AboveAvg.

Quick Solutions (CTEs)

(1–2) DeptAvg + above-average

```
WITH DeptAvg AS (  
    SELECT Dept, AVG(GPA) AS AvgGPA  
    FROM Student  
    GROUP BY Dept  
)  
SELECT s.SID, s.Name, s.Dept, s.GPA  
FROM Student s  
JOIN DeptAvg d ON d.Dept = s.Dept  
WHERE s.GPA > d.AvgGPA;
```

(3) Two CTEs: DeptAvg then AboveAvg

```
WITH  
DeptAvg AS (  
    SELECT Dept, AVG(GPA) AS AvgGPA  
    FROM Student  
    GROUP BY Dept  
) ,  
AboveAvg AS (  
    SELECT s.SID, s.Name, s.Dept, s.GPA  
    FROM Student s  
    JOIN DeptAvg d ON d.Dept = s.Dept  
    WHERE s.GPA > d.AvgGPA  
)  
SELECT *  
FROM AboveAvg  
ORDER BY Dept, GPA DESC;
```

In-class Quiz: Recursive CTEs (hierarchy)

Write the exact SQL code (Org chart).

Table: *Employee*(*EmpID*, *Name*, *ManagerID*) where *ManagerID* references *EmpID*.

- 1 Write a recursive CTE to output the full org chart starting from the CEO (*ManagerID* IS NULL), with a computed *Level* (CEO at level 0).
- 2 Modify it to return *only* the subtree under a given manager (parameter: `:manager_id`).
- 3 Add a *Path* column showing the chain of names from the root to each employee (e.g., CEO > ... > Alice).

Quick Solutions (Recursive CTEs)

(1) Full org chart from CEO + Level

```
WITH RECURSIVE Org AS (  
    SELECT EmpID, Name, ManagerID, 0 AS  
        Level  
    FROM Employee  
    WHERE ManagerID IS NULL  
    UNION ALL  
    SELECT e.EmpID, e.Name, e.ManagerID, o.  
        Level + 1  
    FROM Employee e  
    JOIN Org o ON e.ManagerID = o.EmpID  
)  
SELECT * FROM Org;
```

(2-3) Subtree under :manager_id + Path

```
WITH RECURSIVE Org AS (  
    -- anchor: chosen manager  
    SELECT EmpID, Name, ManagerID,  
        0 AS Level,  
        CAST(Name AS VARCHAR(500)) AS  
            Path  
    FROM Employee  
    WHERE EmpID = :manager_id  
    UNION ALL  
    SELECT e.EmpID, e.Name, e.ManagerID,  
        o.Level + 1,  
        CONCAT(o.Path, ' > ', e.Name)  
    FROM Employee e  
    JOIN Org o ON e.ManagerID = o.EmpID  
)  
SELECT *  
FROM Org  
ORDER BY Level, EmpID;
```

Key takeaways (L32: Advanced SQL)

- ▶ **CTEs (WITH)** help you write SQL in readable, named steps.
- ▶ **Multiple CTEs** let you build a query pipeline.
- ▶ **Recursive CTEs** handle hierarchies (org-charts, trees) directly in SQL.